

PLP - Primer Parcial - 1^{er} cuatrimestre de 2025 - Parte 2

#Orden	Libreta	Apellido y Nombre	Ej1	Ej2	Ej3	Nota Final
		Pérez Natalia	E	E	E	P

(Promocionado)

Las notas para cada ejercicio son: -, I, R, B, MB, E. Este examen se aprueba obteniendo al menos dos ejercicios B y uno R, y se promociona con al menos dos ejercicios MB y uno B. Es posible obtener una aprobación condicional con un ejercicio MB, uno B y uno I. Esto requiere entregar algo que contribuya a la solución del ejercicio. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Programación funcional

Aclaración: en este ejercicio no está permitido utilizar recursión explícita, a menos que se indique lo contrario.

Se define el tipo de datos ABNV a, que representa árboles binarios no vacíos con elementos de tipo a:

`data ABNV a = Hoja a | Uni a (ABNV a) | Bi (ABNV a) a (ABNV a)`

Definimos el siguiente árbol para los ejemplos:

`abnv = Bi (Uni 2 (Hoja 1)) 3 (Bi (Hoja 4) 5 (Uni 2 (Hoja 7)))`

- Definir las funciones `foldABNV` y `recABNV`, que implementan respectivamente los esquemas de recursión estructural y primitiva para el tipo ABNV a. Solo en este inciso se permite usar recursión explícita.
- Definir la función `elemABNV :: Eq a => a -> ABNV a -> Bool`, que indica si un elemento pertenece a un árbol.
Por ejemplo: `elemABNV 7 abnv ~ True`.
- Definir la función `reemplazarUno :: Eq a => a -> a -> ABNV a -> ABNV a` que, dados dos elementos `x` e `y` y un árbol, devuelve un árbol como el original, pero reemplazando la primera aparición de `x` por `y` (la primera desde la raíz yendo de izquierda a derecha, en el orden de preorden).
Por ejemplo:
`reemplazarUno 2 5 abnv ~ Bi (Uni 5 (Hoja 1)) 3 (Bi (Hoja 4) 5 (Uni 2 (Hoja 7)))`.
`reemplazarUno 2 5 (Hoja 1) ~ Hoja 1`.
- Definir la función `nivel :: ABNV a -> Int -> [a]`, que liste todos los elementos del nivel indicado, de izquierda a derecha, siendo 0 el nivel de la raíz. Si el nivel no existe (ya sea por ser negativo o por superar la altura del árbol), devolver `[]`.

Por ejemplo:

`nivel abnv 1 ~ [2,5]`.

`nivel abnv 2 ~ [1,4,2]`.

`nivel abnv 4 ~ []`.

Pista: aprovechar la curificación y utilizar evaluación parcial.

Ejercicio 2 - Demostración de propiedades

- Considerar las siguientes definiciones:

`data AB a = Nil | Bin (AB a) a (AB a)`

`elemAB :: Eq a => a -> AB a -> Bool`

`{P0} elemAB _ Nil = False`

`{P1} elemAB x (Bin i r d) = (x == r) || (elemAB x i) || (elemAB x d)`

`mapAB :: (a -> b) -> AB a -> AB b`

`{M0} mapAB _ Nil = Nil`

`{M1} mapAB f (Bin i r d) = Bin (mapAB f i) (f r) (mapAB f d)`

Sean dos tipos a y b tales que valen $\text{Eq } a$, $\text{Eq } b$, y por ende también la siguiente propiedad:

$$\{\text{CONGRUENCIA} ==\} \forall x::a. \forall y::a. \forall f::a \rightarrow b. x == y \Rightarrow f x == f y$$

Demostrar la siguiente propiedad:

$$\forall t::AB \ a. \forall x::a. \forall f::a \rightarrow b. (\text{elemAB } x \ t \Rightarrow \text{elemAB } (f \ x) \ (\text{mapAB } f \ t))$$

Se permite definir macros (i.e., poner nombres a expresiones largas para no tener que repetirlas).

No es obligatorio escribir los $\forall$ correspondientes en cada paso, pero es importante recordar que están presentes. Recordar también que los $=$ de las definiciones pueden leerse en ambos sentidos.

Se consideran demostradas todas las propiedades conocidas sobre enteros y booleanos.

Sugerencia: tener en cuenta que $\forall x::\text{Bool}. \forall y::\text{Bool}. \forall z::\text{Bool}. (x \Rightarrow y) \Rightarrow (x \Rightarrow z \vee y)$.

b) Demostrar el siguiente teorema usando Deducción Natural, sin utilizar principios clásicos:

$$\rho \Rightarrow (\sigma \vee (\rho \Rightarrow \tau)) \Rightarrow (\sigma \vee \tau)$$

Ejercicio 3 - Cálculo Lambda Tipado

Se desea extender el cálculo lambda simplemente tipado para modelar **listas ordenadas**. Una lista ordenada se construye de manera similar a una lista común, pero sus observadores se comportan de manera diferente: la cabeza de la lista es siempre el mínimo elemento, y su cola es el resto de los elementos.

Estas listas tienen como precondition que el tipo de sus elementos cuente con una operación de comparación $<$ (no es necesario escribir nada al respecto, suponemos que vale).

Se extienden los tipos y expresiones de la siguiente manera:

$$\tau ::= \dots \mid [\tau]^<$$

$$M ::= \dots \mid []_\tau^< \mid M ::^< M \mid \text{head}^<(M) \mid \text{tail}^<(M)$$

donde:

- $[\tau]^<$ es el tipo de las listas ordenadas con elementos de tipo τ .
- $[]_\tau^<$ es una lista ordenada vacía que admite valores de tipo τ .
- $M_1 ::^< M_2$ es la lista resultante de agregar el elemento M_1 a la lista M_2 ¹.
- $\text{head}^<(M)$ denota el mínimo elemento de la lista M .
- $\text{tail}^<(M)$ es la lista ordenada con todos los elementos de M excepto el mínimo (si M tiene más de una aparición de su mínimo elemento, se elimina una de ellas).

a) Introducir las reglas de tipado para la extensión propuesta.

b) Definir el conjunto de valores y las nuevas reglas de reducción en un paso. Tener en cuenta que una lista vacía no tiene cabeza ni cola.

Pista: puede ser necesario mirar más de un nivel de un término para saber a qué reduce.

c) Mostrar paso por paso cómo reduce la expresión:

$$\text{head}^<((\lambda x: \text{Nat}. 2)\text{zero}) ::^< 1 ::^< []_{\text{Nat}}^<$$

Se pueden considerar dadas las siguientes reglas:

$$\frac{M_1 \rightarrow M'}{M_1 < M_2 \rightarrow M' < M_2} (E-<_1) \quad \frac{M_2 \rightarrow M'}{V < M_2 \rightarrow V < M'} (E-<_2)$$

$$\frac{}{V < \text{zero} \rightarrow \text{False}} (E-<_F) \quad \frac{}{\text{zero} < \text{Succ}(V) \rightarrow \text{True}} (E-<_T)$$

$$\frac{}{\text{Succ}(V_1) < \text{Succ}(V_2) \rightarrow V_1 < V_2} (E-<_S)$$

¹Notar que M_1 no necesariamente es la cabeza de la lista, y que $::^<$ es un constructor, no es responsable de ordenar los elementos en la lista. El orden aparece al observarla. Por ejemplo, $1 ::^< 2 ::^< []_{\text{Nat}}^<$ y $2 ::^< 1 ::^< []_{\text{Nat}}^<$ son dos valores de tipo $[\text{Nat}]^<$ que tienen la misma cabeza y la misma cola.